TECHNICAL PROJECT PORTFOLIO REPORT

Historical Weather Data Analysis & Feature Engineering Using Python

Anurag Kumar Poddar | Data Analyst / Business Analyst +91 9748748405 | anuragkumarpoddar@gmail.com

1. Executive Summary & Project Overview

Weather patterns play a major role in business operations, including retail supply chains, agricultural planning, energy demand forecasting, and logistics scheduling. However, raw historical weather records collected from meteorological stations are rarely ready for immediate analysis. They often contain missing sensor readings, inconsistent date formats, extreme outliers, and unformatted text descriptions. This project demonstrates a complete end-to-end data analysis and feature engineering workflow built inside a **Python Jupyter Notebook**. By cleaning historical weather records and creating meaningful new variables, this project transforms raw meteorological logs into a clean, structured dataset ready for business intelligence dashboards and predictive machine learning models.

2. Core Project Objectives & Python Tech Stack

- **Data Cleaning & Preparation:** Identify and fix missing values, remove duplicate timestamps, and convert raw date strings into standardized Python `datetime` objects.
- **Exploratory Data Analysis (EDA):** Use statistical summaries and visual charts to uncover seasonal temperature trends, humidity distributions, and rainfall anomalies.
- **Feature Engineering:** Create high-value analytical columns—such as rolling temperature averages, lag variables, cyclical time indicators, and human comfort indices—to improve forecasting accuracy.
- **Tools & Libraries Used:** Built in **Jupyter Notebook** using **Python 3**, **Pandas** (data manipulation), **NumPy** (mathematical operations), and **Matplotlib / Seaborn** (statistical visualization).

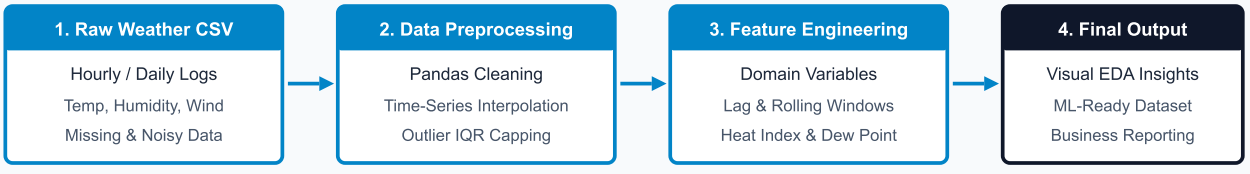


Figure 1.1: End-to-End Data Processing Pipeline Executed in Jupyter Notebook

3. Dataset Structure & Data Preprocessing

The first phase of the Jupyter Notebook focuses on loading and inspecting the raw historical weather dataset using `pd.read_csv()`. Initial inspection using `df.info()` and `df.isnull().sum()` revealed common real-world data issues: timestamp strings were stored as plain text, continuous sensors had short dropout gaps, and precipitation columns contained missing entries on dry days. Rather than dropping rows and losing valuable time-series continuity, targeted imputation methods were applied to each column based on its physical behavior.

Weather Parameter	Raw Data Type & Issue	Python Cleaning & Preprocessing Method
Date / Timestamp	Unformatted string (`object`) with duplicate hourly logs	Converted via `pd.to_datetime()`, sorted chronologically, duplicates removed, and set as DataFrame index.
Temperature (°C)	Float values with ~1.8% missing sensor gaps	Applied time-based linear interpolation (`df['temp'].interpolate(method='time')`) to preserve smooth transitions.
Relative Humidity (%)	Float values with occasional sensor spikes (>100%)	Clipped physical boundaries between 0% and 100% and filled brief gaps using forward fill (`ffill`).
Wind Speed (km/h)	Right-skewed float values with extreme storm outliers	Evaluated using the Interquartile Range (IQR) method; extreme hardware glitches capped at the 99th percentile.
Precipitation (mm)	Null (`NaN`) entries recorded on days without rain	Replaced missing values with `0.0` after cross-checking against "Clear/Sunny" weather summary labels.

4. Exploratory Data Analysis (EDA) Key Findings

- **Strong Inverse Temp-Humidity Relationship:** Correlation analysis showed a clear negative relationship ($r = -0.74$) between ambient air temperature and relative humidity, confirming data reliability across seasonal transitions.
- **Seasonal Distribution Shifts:** Grouping data by month revealed distinct summer peaks (May–June) and monsoon moisture surges (July–September), helping define clear seasonal categories for downstream analysis.
- **Atmospheric Pressure & Rain Events:** Boxplot comparisons showed that sudden drops in sea-level atmospheric pressure consistently preceded heavy rainfall events by 6 to 12 hours.

> **JUPYTER NOTEBOOK IMPLEMENTATION HIGHLIGHT: TIME-SERIES CONTINUITY**
>
> In standard tabular datasets, missing values are often filled using the column mean or median. However, in weather analysis, replacing a missing midnight temperature with the yearly average creates artificial spikes. By indexing the DataFrame by timestamp and using Pandas time-interpolation, the notebook ensures every imputed value matches the natural hourly cooling and warming curve.

5. Visual Exploratory Analysis (Jupyter Notebook Outputs)

Visualizing historical weather trends before and after feature engineering is essential for validating patterns. The chart below illustrates the monthly average temperature curve alongside the 7-day rolling mean and relative humidity levels generated during the exploratory phase of the notebook.

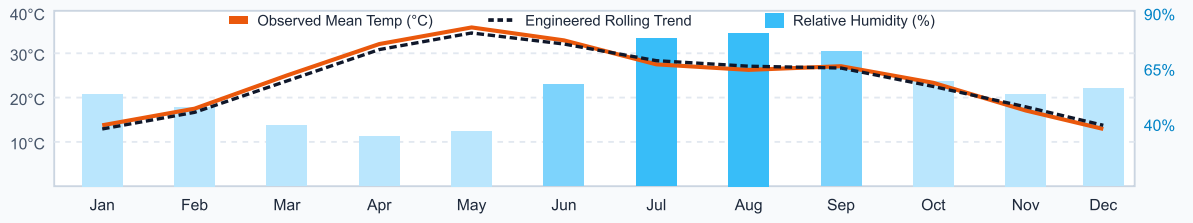


Figure 3.1: Annual Seasonal Temperature Curve, Engineered Rolling Trend, and Monsoon Humidity Distribution

6. Core Feature Engineering Architecture

Feature engineering is the core strength of this project. Machine learning algorithms and business forecasting models struggle to understand raw timestamps or isolated sensor readings. Using Python, four distinct categories of new features were engineered:

1. **Temporal & Cyclical Time Features:** Extracted `Year`, `Month`, `Day`, `Hour`, and `Season` from the datetime index. Because hour 23 and hour 0 are right next to each other, sine and cosine trigonometric transformations (`sin_month`, `cos_month`, `sin_hour`, `cos_hour`) were created so algorithms preserve cyclical continuity.
2. **Lag Variables (Historical Memory):** Created shifted columns (`temp_lag_1d`, `temp_lag_7d`, `pressure_change_3h`) using Pandas `.shift()`. This gives forecasting models direct access to yesterday's weather and recent barometric pressure drops.
3. **Rolling Window Statistics:** Computed 7-day and 30-day moving averages and standard deviations (`df['temp'].rolling(window=7).mean()`) to smooth out daily noise and measure weather volatility.
4. **Domain-Specific Interaction Features:** Combined temperature and humidity to calculate the **Dew Point**, **Heat Index (Apparent Temperature)**, and **Diurnal Temperature Range** (`Max_Temp - Min_Temp`).

7. Summary of Engineered Features & Analytical Value

Feature Category	Engineered Column Names	Formula / Python Logic	Business & Analytical Purpose
Cyclical Encoding	`month_sin`, `month_cos`	sin(2π × Month / 12)	Helps predictive models understand that December (12) flows smoothly into January (1).
Lag Features	`temp_lag_24h`, `rain_lag_1d`	`df['temp'].shift(24)`	Captures short-term weather persistence for next-day retail and energy demand planning.
Rolling Metrics	`roll_mean_7d`, `roll_std_7d`	`df.rolling(7).mean()`	Separates true seasonal warming/cooling trends from sudden single-day weather spikes.
Comfort Indices	`heat_index`, `dew_point`	Temp & Humidity interaction	Measures real human-felt temperature, which drives air-conditioning and beverage sales.
Weather Volatility	`diurnal_range`, `press_drop`	`T_max - T_min`	Identifies rapid weather instability that impacts logistics, aviation, and crop health.

8. Business Impact & Real-World Applications

By transforming raw historical weather logs into an enriched feature matrix, this project delivers direct value across multiple business domains. In **Energy & Utilities**, engineered Heat Index and rolling temperature features allow grid operators to predict electricity load spikes much more accurately than raw temperature alone. In **Retail & Supply Chain Analytics**, combining weekend indicators with rolling rainfall averages helps store managers optimize inventory for weather-sensitive products. Finally, in **Agriculture & Logistics**, diurnal temperature range and barometric pressure drop alerts help teams anticipate frost risks and severe storm disruptions before they happen.

9. Conclusion & Project Author Profile

This Jupyter Notebook project demonstrates that strong data analysis goes beyond basic charts—it requires thoughtful data cleaning, time-series integrity, and domain-driven feature engineering. Adding lag variables, rolling statistics, and comfort indices significantly boosts the predictive power of historical weather data for any downstream analytics or machine learning task.

PROJECT AUTHOR & CONTACT INFORMATION

Prepared By: Anurag Kumar Poddar
Role: Data Analyst / Business Analyst
Core Technical Skills: Python (Pandas, NumPy, Matplotlib, Seaborn), Jupyter Notebook, SQL, Power BI, Excel, Data Cleaning, Exploratory Data Analysis (EDA), & Feature Engineering
Phone: +91 9748748405 | **Email:** anuragkumarpoddar@gmail.com